A Generic Representation of Program Organization

Brandon Bosman, Dr. Jacques Carette
Department of Computing and Software, McMaster University

McMaster University | Computing & Software

What Is GOOL?

GOOL, the Generic Object-Oriented Language, is a language that is generic over mainstream object-oriented languages [1]. GOOL takes a "write once, generate everywhere" approach, meaning that a program written in GOOL can initially translated to one language, then translated to another language with few or no changes if the first language is found lacking.

Program Organization

All mainstream languages have some mechanism for organizing a piece of software and managing visibility of names. This usually involves a tree of directory-like things and file-like things. There are many differences between languages, however, e.g.:

- Java: packages with public/private, no subpackages [2]
- C#: namespaces with public/private, with nested namespaces
- Swift: modules (i.e. files) with visibility management
- Python, Julia: packages represented as files, no visibility management options
- Rust: modules and submodules (usually folders and subfolders)
- .... etc. Every language has its quirks.

Problem: current GOOL doesn't know about these differences!

- It assumes that a program is simply a list of files, with no other nuances.

Generically Representing Program Organization in GOOL

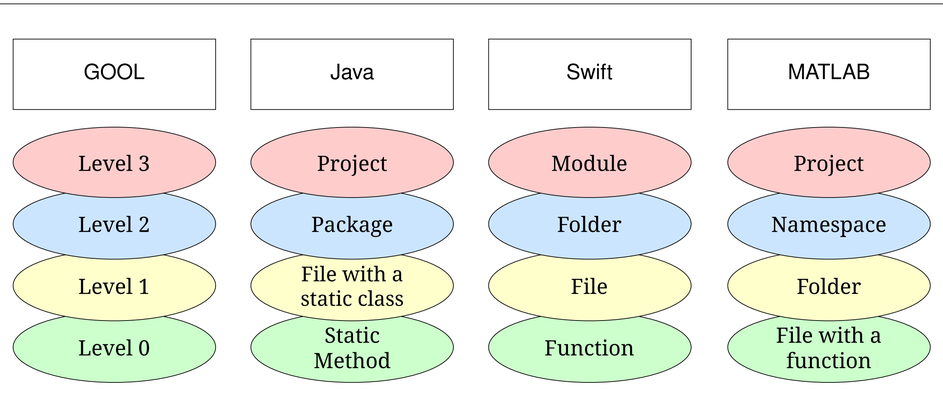
GOOL aims to be a "write once, generate everywhere" language: one GOOL program should be usable in generating a host of idiomatic code in a variety of desired languages, with no language-specific concerns. This requires a generic representation of program organization that maps onto each language's own structures.

Common terms like "package", "module", and "namespace" mean drastically different things across languages; we avoid their use entirely in GOOL and use indexed levels of abstraction instead.

I need to put something here...







Relation to Visibility

On top of pure organization, these "organizational structures" are also used to control which names (variables, functions, types, etc.) are accessible from other "organizational structures". The semantics of this also varies between languages, so we need to make sure we translate visibility management in a correct and idiomatic manner. If a language's vocabulary cannot express what we want to express safely (i.e. cannot prevent a name collision or cannot make one name reachable from a structure that needs it), then we should detect this and throw an error.

References

- [1] Carette, J., MacLachlan, B., Smith, S. (2020). GOOL: A Generic Object-Oriented Language. PEPM '20. https://dl.acm.org/doi/10.1145/3372884.3373159

Acknowledgements

Mention OGS and NSERC?